



SUPPORT FORMATEUR · BRANCHE JOUR3-CORRIGE

Formation Prometheus & Grafana

Jour 3 — corriger et démontrer : les étapes, ce que je montre, ce que je dis, les réponses

Sommaire

Jour 3 — Corriger et démontrer avec la branche jour3-corrige	2
Avant de corriger — 5 minutes	2
Correction du TP A — La boutique en quatre signaux dorés	3
Correction du TP B — Le serveur en méthode USE	5
Auto-audit — ce que je montre	6
Correction du TP C — Prometheus, Alertmanager, Teams, Slack, e-mail	6
Correction du TP D — L'alerting de Grafana	8
Correction de l'escape game	9
Et après la correction	10
Annexe — Les commandes de démonstration	10

Jour 3 — Corriger et démontrer avec la branche jour3-corrige

Ce document se suit à l'écran, projeté ou sur un second écran, pendant les corrections. Pour chaque TP : **ce que j'ouvre**, **ce que je montre** (clic par clic), **ce que je dis** (en citations) et **les réponses** aux questions du guide stagiaire. Tout se fait dans mon Codespace sur la branche `jour3-corrige`, où tous les TP sont déjà faits.

La règle d'or de la correction : je ne montre jamais un panel sans **casser la boutique** pour le voir réagir. « Un dashboard qu'on n'a jamais vu réagir à une panne, on ne sait pas s'il marche. »

Avant de corriger — 5 minutes

Pas à pas — ouvrir le Codespace corrigé

1. Si la branche n'est pas encore poussée, depuis mon Mac (Terminal) : `bash`
`" /Users/yparent/Documents/PERSO/YSYCloud/FORMATION/PROMETHEUS GRAFANA/2026/POUSSER-JOUR3-CORRIGE.command "`
2. Dans le navigateur : <https://codespaces.new/yparent/formation-observabilite-lab/tree/jour3-corrige> → vérifier la branche `jour3-corrige` → **Create codespace**.
3. Dans le terminal du Codespace : `./jour3.sh start` . J'attends `10 / 10 cibles UP` .
4. J'ouvre ces onglets, dans cet ordre (onglet **PORTS**, icône globe) :

Onglet	Port	Pour
Grafana	3000	<code>admin</code> / <code>formation</code> ; les dashboards et l'alerting Grafana
Prometheus	9090	Alerts, Status → Target health
Alertmanager	9093	les alertes reçues, <i>Inhibited</i> , New Silence
Inbox	8080	les cartes « Teams » et les messages « Slack »
Mailpit	8025	les e-mails

5. Dans Grafana : **Dashboards → Formation**. Deux dashboards : **Boutique - Signaux dorés** et **Serveur - USE**. Je règle le rafraîchissement automatique sur **10s** (en haut à droite).

Astuce de projection : j'agrandis la police du navigateur (Cmd +) à 110 %, et je masque la barre latérale de Grafana (icône en haut à gauche) pour gagner de la place.

Correction du TP A — La boutique en quatre signaux dorés 15 min

Ce que j'ouvre

Grafana → **Dashboards** → **Formation** → **Boutique - Signaux dorés**. Période : *Last 15 minutes*.

Ce que je dis d'abord 1 min

« Voilà un dashboard construit avec une méthode. Lisez-le de haut en bas, comme un immeuble. En haut, le panel texte : à quoi sert ce dashboard, qui contacter. Puis le rez-de-chaussée : quatre cases, les quatre signaux dorés de Google, et la réponse à "ça va ?" en cinq secondes. Au premier, les mêmes signaux dans le temps. Au deuxième, ce qu'on vend. »

Pas à pas — la row « Est-ce que ça va ? » 5 min

Pour chaque case, je survole le panel →  (en haut à droite du panel) → **Edit** (ou touche **e**). Je montre la requête en bas, puis les réglages à droite. **Back to dashboard** ensuite.

Case	Ce que je montre dans l'éditeur	Ce que je dis
Trafic	<code>sum(rate(http_requests_total{...}[\$__rate_interval])) ; Unit : requests/sec ; Standard options → Color scheme : Single color bleu</code>	« Le trafic est en bleu, jamais en rouge : un afflux de clients, c'est une bonne nouvelle. »
Erreurs	<code>(sum(rate(...{status=~"5.."}...)) or vector(0)) / sum(rate(...)) ; Unit : Percent (0.0-1.0) ; seuils 0.01 orange, 0.05 rouge</code>	« Le <code>or vector(0)</code> : sans lui, tant qu'aucune erreur n'a jamais eu lieu, la case affiche "No data". Avez-vous eu ce "No data" ce matin ? Voilà pourquoi : la série des 5xx n'existe pas encore. »
Latence p95	<code>histogram_quantile(0.95, sum by (le) (rate(..._bucket[...]))) ; Unit : seconds ; seuils 0.5 / 1</code>	« Le <code>le</code> doit rester dans le <code>by</code> : c'est l'escalier des buckets. Sans lui, "No data". »
Saturation	<code>100 * (1 - avg(rate(node_cpu_seconds_total{mode="idle"}...))) ; Unit : Percent (0-100) ; seuils 70 / 90</code>	« Pas de filtre <code>instance</code> ici : la variable liste les instances de la boutique, pas la machine. Le CPU, c'est le Node Exporter, <code>job="node"</code> . »

Réglages communs, à montrer une fois : *Stat styles* → *Color mode* : **Background** ; *Graph mode* : **Area** (la petite courbe dans la case).

Pas à pas — la démonstration qui fait tout comprendre 3 min

1. Terminal : `./lab.sh chaos errors on`.
2. Je reviens sur le dashboard et je ne touche plus à rien. Je compte à voix haute.

- En 30 secondes : un **trait rouge vertical** apparaît sur toutes les courbes (l'annotation *Chaos*). En 1 à 2 minutes : la case **Erreurs** passe à l'orange puis au **rouge** (30 à 40 %), la courbe *Taux d'erreur* entre dans la zone rouge.
- Terminal : `./lab.sh chaos latency on`. La case **Latence p95** devient rouge, la **heatmap** monte d'un bloc vers le haut.
- `./lab.sh chaos errors off` et `./lab.sh chaos latency off`.

« Regardez la case Erreurs : elle a mis plus d'une minute à rougir. C'est la réponse à votre question de l'étape 1. »

Les réponses aux questions du TP A

Étape 1 — Le signal qui a réagi le plus lentement. Les erreurs et la latence. Elles sont calculées avec `rate()` sur une fenêtre (`$__rate_interval`, au moins une minute) : au début de la panne, la fenêtre contient encore surtout des minutes saines. Le CPU réagit dès le scrape suivant. Fenêtre courte : réactive mais elle clignote ; fenêtre longue : stable mais lente. Le même compromis que le `for` des alertes cet après-midi.

Étape 3 — Pourquoi `rate` pour un montant ? `shop_revenue_euros_total` est un compteur : tout ce qui a été vendu depuis le démarrage, remis à zéro à chaque redémarrage. `rate()` × 3600 donne la **vitesse** en euros par heure. « Le compteur kilométrique contre le compteur de vitesse. »

Étape 5 — Le crash test. Démonstration : `./lab.sh traffic 30`. Le trafic, la saturation et le chiffre d'affaires montent ; erreurs et latence restent vertes. « **Ce n'est pas une panne** : c'est le Black Friday qui se passe bien. » Puis `./lab.sh traffic 6`.

Pas à pas — les finitions 3 min

- Le panel Text** : *Edit* → à droite, le contenu Markdown. « Trois lignes pour la personne d'astreinte qui arrive ici à 3 h du matin. »
- L'annotation** : roue dentée **Settings** (en haut) → **Annotations** → *Chaos* → la requête `changes(shop_chaos_mode{instance=~"$instance"}[1m]) > 0`. « Dès que le mode chaos change, un trait. En production : vos déploiements. »
- Les liens** : en haut du dashboard, le menu **Formation**. Je clique : il mène au dashboard *Serveur - USE, sur la même période*. Dans **Settings** → **Links** : type *Dashboards*, tag `formation`, *Include current time range*. « Un lien par tag : tout dashboard qui portera le tag `formation` apparaîtra dans ce menu, sans rien modifier. »
- La variable** : en haut à gauche, **Instance** → je choisis `shop-api-1` seulement. Tous les panels de la boutique se filtrent. « Un dashboard pour 2 instances ou pour 200. »

Les erreurs que j'ai vues ce matin, et la correction

Ce qu'ils ont eu	Pourquoi	La correction
« No data » sur Erreurs	aucune erreur depuis le démarrage	<code>or vector(0)</code> au numérateur

Ce qu'ils ont eu	Pourquoi	La correction
« No data » sur Saturation	filtre <code>instance=~"\$instance"</code> sur une métrique <code>node_...</code>	retirer le filtre dans le dashboard boutique
Des milliers de requêtes	compteur brut, sans <code>rate</code>	<code>rate(...[\$__rate_interval])</code>
Latence vide	<code>sum by (route)</code> sans <code>le</code>	<code>sum by (le)</code>
Heatmap grise	<i>Format : Heatmap</i> oublié	sous la requête : <i>Options</i> → <i>Format</i> → <i>Heatmap</i>
Stock en 12 barres	une barre par instance	<code>avg by (product)</code>

Correction du TP B — Le serveur en méthode USE 10 min

Ce que j'ouvre

Le menu **Formation** en haut du dashboard boutique → **Serveur - USE** (je montre que le lien garde la période).

Ce que je dis 1 min

« Trois questions par ressource : est-elle occupée ? Est-ce que du travail attend ? Y a-t-il des erreurs ? Une row par ressource : CPU, mémoire, disque et réseau, puis la disponibilité. »

Pas à pas — les trois panels à expliquer

1. **CPU · Saturation (charge / cœur)** → *Edit*. La requête : `node_load1{...} / on (instance) count by (instance) (node_cpu_seconds_total{mode="idle", ...})`.

« C'était le piège de l'IA. À gauche, une série avec `instance` et `job` ; à droite, un `count` sans aucun label. PromQL associe les séries qui ont les mêmes labels : rien ne correspondait, d'où le "No data". On garde `instance` des deux côtés avec `by (instance)`, et on dit sur quoi associer avec `on (instance)`. Une valeur de 1, c'est tous les cœurs occupés ; au-delà, du travail attend. »

2. **CPU · Saturation (pression PSI)** : « Le noyau Linux mesure directement le temps pendant lequel des tâches attendent le CPU. La saturation à l'état pur. »
3. **Mémoire · Utilisation** → la requête avec `MemAvailable`. « Pas `MemFree` : Linux se sert de la mémoire libre comme cache disque et la rend à la demande. `MemFree` est toujours bas et fait peur pour rien. »

Pas à pas — la chaîne de diagnostic 3 min

1. Terminal : `./lab.sh chaos cpu 120`.

2. Dashboard **Boutique** : la case **Saturation** passe à l'orange ou au rouge.
3. Menu **Formation** → **Serveur - USE** : la jauge CPU monte, la charge par cœur dépasse 1, la pression PSI grimpe.

« Un symptôme en haut, un clic, la cause en bas. C'est exactement l'usage des deux méthodes : on alerte avec les signaux dorés, on diagnostique avec USE. »

Les réponses

- **Panel 2 « No data »** : voir ci-dessus (`on (instance)` et `by (instance)`).
- **1860 ou le vôtre ?** À 3 h du matin, le vôtre : peu de panels, des couleurs, la réponse en cinq secondes. Pour une enquête de deux heures, le 1860.

Auto-audit — ce que je montre 3 min

Sur le dashboard Boutique corrigé, je reprends la grille de dix critères et je coche à voix haute : une question et un public (le panel Text) ; « ça va ? » en haut ; une méthode (signaux dorés) ; des unités partout ; un rouge cohérent ; 16 panels mais répartis en rows repliables ; une variable ; les annotations ; les liens par tag ; rangé dans *Formation* et **en code**.

« Ce dashboard est dans le dépôt, en JSON : `grafana/dashboards/boutique-signaux-dores.json` . Grafana l'a chargé tout seul au démarrage. C'est le niveau "élevé" du modèle de maturité : le dashboard est du code. »

Je le montre : dans l'explorateur du Codespace, j'ouvre `grafana/dashboards/boutique-signaux-dores.json` .

Correction du TP C — Prometheus, Alertmanager, Teams, Slack, e-mail 12 min

Pas à pas — les règles 3 min

1. Dans le Codespace, j'ouvre `prometheus/rules/alerts.yml` . Je lis `ShopHighErrorRate` : l'expression, `for: 1m` , les labels `severity: critical` et `team: boutique` , les annotations.
2. Prometheus (9090) → **Alerts** : les six règles, toutes *Inactive* (vertes).

Les réponses de la partie 1 :

1. Symptômes : `ShopHighErrorRate` , `ShopCheckoutSlow` , `BlackboxProbeFailed` , `TargetDown` . Cause : `HostHighCpuLoad` (un CPU haut peut être un serveur qui travaille bien).
2. `for: 1m` : la condition doit tenir une minute avant de notifier (*Pending* → *Firing*). `team` : ne sert qu'au **routage** dans l'Alertmanager.

3. Réveillent (critical, donc Teams) : `ShopHighErrorRate` , `TargetDown` , `BlackboxProbeFailed` . Ne doit jamais réveiller : `HostHighCpuLoad` .

Pas à pas — le routage 3 min

1. Terminal : `docker compose exec alertmanager amtool config routes` . L'arbre s'affiche.
2. Je le dessine au tableau : racine `inbox-default` ; `severity=critical` → `astreinte-teams` (`continue:true`) ; `team=boutique` → `boutique` (Slack + e-mail) ; `team=infra` → `infra-inbox` .
3. Je fais deviner, puis je tape : `docker compose exec alertmanager amtool config routes test severity=critical team=boutique` → `astreinte-teams,boutique` .

Labels	Réponse d'amtool
<code>severity=critical team=boutique</code>	<code>astreinte-teams, boutique</code>
<code>severity=warning team=boutique</code>	<code>boutique</code>
<code>severity=warning team=infra</code>	<code>infra-inbox</code>
<code>severity=critical</code>	<code>astreinte-teams</code>
<code>severity=info team=logistique</code>	<code>inbox-default</code>

« Sans le `continue` , l'astreinte est prévenue mais pas l'équipe boutique. L'erreur de routage la plus fréquente en production. »

Pas à pas — casser et suivre 4 min

1. Je mets côte à côte : Prometheus **Alerts**, Inbox (8080), Mailpit (8025).
2. Terminal : `./lab.sh chaos errors on` . Je note l'heure au tableau.
3. Prometheus → **Alerts** : `ShopHighErrorRate` passe *Pending* (orange) en 15 à 30 s, puis *Firing* (rouge) une minute plus tard.
4. 10 secondes après : dans l'**Inbox**, une carte **Teams** (astreinte) et un message **Slack** (boutique). Dans **Mailpit**, un e-mail `[FIRING:2] ShopHighErrorRate` . Je l'ouvre.

« Deux instances en panne, **un seul** message : c'est le regroupement, `group_by: [alertname, job]` . »

5. Sans arrêter les erreurs : `./lab.sh chaos latency on` . Deux minutes plus tard : `docker compose exec alertmanager amtool alert query --inhibited` → `ShopCheckoutSlow` **suppressed**. Rien dans l'**Inbox** ni dans Mailpit pour elle.

« L'inhibition : le site est déjà en erreur, l'astreinte est réveillée, inutile d'ajouter "le paiement est lent". Et elle ne joue que si l'alerte critique est déjà active. »

6. **Le silence** : Alertmanager (9093) → **New Silence** → matcher `team="boutique"` , 30 min, commentaire « Maintenance ». « L'inhibition est une règle permanente ; le silence, une décision humaine, avec un nom et une durée. » Puis **Expire**.

7. `./jour3.sh repare` .

Pas à pas — le vrai Teams 2 min, si pas déjà fait

1. Terminal : `./jour3.sh teams 'ADRESSE-DU-WORKFLOW'` (entre apostrophes).
2. `./lab.sh chaos errors on` → la carte arrive dans le canal Teams projeté en 1 à 2 minutes.
3. `./lab.sh chaos errors off` , puis `./jour3.sh teams off` .

Correction du TP D — L'alerting de Grafana 10 min

Pas à pas — les objets provisionnés 4 min

1. Grafana → **Alerting** → **Notification configuration** → **Contact points** : `equipe-commerce` (Email) et `teams-salle` (Microsoft Teams), avec la mention **Provisioned**.

« Ils viennent d'un fichier, `grafana/provisioning/alerting/jour3-corrige.yml` : l'alerting Grafana aussi peut être du code. Contrepartie : un objet provisionné ne se modifie pas dans l'interface. »

2. Onglet **Notification policies** : la politique par défaut → `teams-salle` ; la politique enfant `team = commerce` → `equipe-commerce` , *Group wait* 10 s, *mute timing* `week-end` .
3. Onglet **Time intervals** : `week-end` , samedi et dimanche, Europe/Paris.
4. **Alerting** → **Alert rules** → **Formation** → **commerce** → **Chiffre d'affaires en chute** → **View** (ou **Edit**) : la requête A (`sum(rate(shop_revenue_euros_total{job="shop-api"}[2m])) * 3600` , **Instant**), le seuil B **IS BELOW 90 000**, *Pending period* 1 min, les labels `team=commerce` , `severity=warning` , le résumé avec `{{ humanize (index $values "A").Value }}` .

Pas à pas — la déclencher 4 min

1. Terminal : `./lab.sh traffic 1` .
2. Je reste sur la règle : *Normal* → *Pending* → **Firing** en 2 à 3 minutes. Pendant l'attente, je fais le tableau comparatif (ci-dessous).
3. **Mailpit** : l'e-mail `[FIRING:1] Chiffre d'affaires en chute` . Je l'ouvre : le résumé affiche le chiffre d'affaires réel.
4. `./lab.sh traffic 6` . Quelques minutes plus tard, l'e-mail `[RESOLVED]` .

Le vrai Teams depuis Grafana 2 min

Contact points → **+ Create contact point**, nom `teams-demo` , intégration **Microsoft Teams**, URL : l'adresse du workflow → **Test** → la carte arrive dans Teams. (Je ne le sauvegarde pas, ou je le supprime après.)

Le tableau comparatif — les réponses

	Prometheus + Alertmanager	Grafana
Où vit la configuration ?	des fichiers YAML	la base de Grafana, ou des fichiers de provisioning
Versionner, relire	Git, merge request : naturellement	export, provisioning, Git Sync : possible
Tester avant de déployer	<code>promtool test rules</code> , <code>amtool config routes test</code>	<i>Preview</i> , <i>Test</i> sur le contact point
Qui peut la modifier ?	ceux qui touchent au YAML	quiconque a les droits sur le dossier
Plusieurs sources (SQL, logs) ?	non	oui
Si Grafana tombe ?	les alertes continuent	plus d'alertes
Pour quoi chez moi ?	les alertes techniques, critiques	les alertes métier, multi-sources

« Les deux coexistent très bien. Une seule règle : jamais la même alerte des deux côtés. »

Correction de l'escape game 5 min

Chacun a lancé `./jour3.sh solution`. Je projette ce tableau et je fais lever les mains par sabotage : « Qui avait le 1 ? Qu'avez-vous vu ? »

N°	Sabotage	Où ça se voit	Alerte et canal
1	Latence ×10 sur <code>shop-api-1</code>	case Latence, heatmap ; variable sur <code>shop-api-1</code>	<code>ShopCheckoutSlow</code> → Slack + e-mail (<i>inhibée</i> si le 2 est actif)
2	40 % d'erreurs sur <code>shop-api-2</code>	case Erreurs (15-20 % au global, 30-40 % sur <code>shop-api-2</code>)	<code>ShopHighErrorRate</code> → Teams + Slack + e-mail
3	Redis arrêté	Explore : <code>redis_up</code> = 0 (<code>up{job="redis"}</code> reste à 1)	aucune : il manque <code>redis_up == 0</code>
4	Trafic ×5	Trafic, Saturation, chiffre d'affaires en hausse	pas une panne
5	CPU saturé 15 min	Saturation, dashboard USE	<code>HostHighCpuLoad</code> → infra-inbox
6	<code>shop-api-1</code> arrêtée	<code>up</code> = 0, <code>probe_success</code> = 0, trafic mesuré divisé par deux	<code>TargetDown</code> → Teams + Slack + e-mail ; <code>BlackboxProbeFailed</code> → Teams
7	Fuite mémoire	Explore : <code>process_resident_memory_bytes{job="shop-api"}</code>	aucune : il manque une alerte <code>predict_linear</code>

Pour montrer le 3 en direct : `docker compose stop redis`, puis Grafana → **Explore** → `up{job="redis"}` (1) et `redis_up` (0). « L'exporter va bien, c'est Redis qui est mort : `up` ne dit que "j'arrive à lire l'exporter". » Puis `./jour3.sh repare`.

Les questions éclair (1 point chacune) :

1. un compteur, on lit sa vitesse avec `rate()` ;
2. la moyenne ne décrit personne, le p95 est une promesse ; 3. le `for` évite les faux positifs ;
3. `continue: true` fait recevoir l'alerte par plusieurs receivers ; 5. inhibition = règle permanente, silence = décision humaine ponctuelle ; 6. l'adresse Teams est un secret, le YAML va dans Git ; 7. `MemAvailable`, car Linux utilise la mémoire libre comme cache ; 8. SQL : Grafana ; survivre à Grafana : Prometheus et Alertmanager.

Et après la correction

- Je dépose dans le chat : `docs/stagiaire/Guide-stagiaire-Jour-3-corriges.pdf` (sur la branche `jour3`, dans leur Codespace) et le lien de la branche corrigée : <https://github.com/yparent/formation-observabilite-lab/tree/jour3-corrige>, pour qu'ils puissent récupérer les dashboards en JSON.
- Pour récupérer les dashboards corrigés dans **leur** Codespace :

```
git fetch origin
git checkout origin/jour3-corrige -- grafana/dashboards/
```

Dix secondes plus tard, les deux dashboards corrigés apparaissent dans leur dossier *Formation*.

- Après chaque démonstration : `./jour3.sh repare`.

Annexe — Les commandes de démonstration

Je veux montrer...	Commande	Où regarder
des erreurs	<code>./lab.sh chaos errors on / off</code>	cases Erreurs, Inbox, Mailpit
de la lenteur	<code>./lab.sh chaos latency on / off</code>	Latence, heatmap, inhibition
un CPU saturé	<code>./lab.sh chaos cpu 120</code>	Saturation, dashboard USE
un afflux de clients	<code>./lab.sh traffic 30 / traffic 6</code>	Trafic, chiffre d'affaires
une chute des ventes	<code>./lab.sh traffic 1 / traffic 6</code>	règle Grafana, Mailpit
une instance tombée	<code>docker compose stop shop-api-1 / ./jour3.sh repare</code>	Cibles, <code>TargetDown</code>
Redis tombé	<code>docker compose stop redis / ./jour3.sh repare</code>	Explore <code>redis_up</code>
le vrai Teams	<code>./jour3.sh teams 'ADRESSE' / teams off</code>	canal Teams

Je veux montrer...	Commande	Où regarder
tout remettre en ordre	<code>./jour3.sh repare</code>	
